

Progress Update: Continual Learning under Visual Drift

1. Where the Project Stands

I have now completed all of the experiments for this project, including the main method comparison, the multi-seed runs, the buffer-size study, and the computational cost analysis.

The project is now in the final thesis-writing stage. I am currently using the completed results to finish the Overleaf draft, refine the discussion, organise the figures and tables, and make sure the experimental design and conclusions are explained clearly.

2. Methods Compared

I compare six methods. I chose them to cover the main families of continual learning approaches rather than only comparing small variations of the same idea.

Method	Type	Why It Is Included
Vanilla fine-tuning	Baseline	Shows how much forgetting happens without any protection
EWC	Regularisation-based	A standard parameter-importance method based on Fisher Information
MAS	Regularisation-based	Another importance-based method, but based on output sensitivity
OGD	Optimisation-based	Represents gradient-projection methods
ER	Replay-based	The simplest direct replay method, useful as a strong practical baseline
A-GEM	Replay + gradient constraint	Uses memory differently from ER, so it helps test whether replay itself or the way memory is used matters

The comparison is designed around one larger question: whether directly revisiting old examples is more useful than only protecting parameters or constraining gradients.

3. Datasets and What Each One Tests

I use three datasets because they create different kinds of drift.

Rotated MNIST has 10 tasks, where the same digit classes are gradually rotated from 0 degrees to 90 degrees. This is a smooth geometric drift setting. Since the label space stays the same, the main challenge is whether the model can keep recognising digits as the input distribution shifts.

EMNIST has 5 tasks with the same 36 character classes, but each task applies a different visual transformation such as rotation, blur, brightness change, or mixed augmentation. I originally expected this to behave like another visual-drift benchmark, but the results show that it is different: because the tasks share many features, later tasks can actually improve earlier performance. This makes EMNIST more of a positive-transfer/plasticity setting than a forgetting-heavy setting.

CIFAR-100 has 10 tasks, each containing 10 fine-grained object classes. This is the hardest setting because the tasks introduce new semantic categories. I use task-incremental evaluation so that the comparison focuses on forgetting and retention rather than making the class-incremental problem too close to chance.

4. Why I Use Three Seeds

I use three random seeds, 0, 1, and 2, because a single continual learning run can be misleading.

There are several sources of randomness in these experiments: the initial model weights, the mini-batch order, replay-buffer sampling, and in some cases augmentation or memory selection. These sources of randomness can matter a lot in continual learning. For example, a replay method may perform differently depending on which examples are stored early in training. A gradient-projection method may also be sensitive to the exact optimisation path.

Using three seeds lets me report **mean $\pm$ standard deviation** instead of relying on one run. The mean gives a better estimate of the method's typical behaviour, while the standard deviation tells me whether the method is stable. This is important because a method that performs well once but varies a lot across seeds is not as convincing as a method that is consistently strong.

This turned out to be especially useful in the buffer-size study. ER stays quite stable across seeds, while A-GEM becomes much less stable on CIFAR-100 when the memory size is large. If I had only run one seed, I might have missed that instability or over-interpreted a lucky run.

5. Why I Compare Several Buffer Sizes

For replay-based methods, memory size is not a small detail. It is one of the main factors that determines what the method is allowed to remember.

That is why I compare buffer or memory sizes of **200, 500, 1000, and 2000** for ER and A-GEM. I use 1000 as the main setting so that ER and A-GEM are compared under the same memory budget, but the extra buffer-size experiments help explain whether the result depends on that one choice.

There are three reasons this matters.

First, it checks whether ER's advantage is robust. If ER only worked well at one buffer size, the conclusion would be weaker. But the results show that ER improves steadily as the buffer gets larger on Rotated MNIST and CIFAR-100.

Second, it helps compare how ER and A-GEM actually use memory. ER reuses stored examples directly during training, so increasing the buffer gives it better coverage of old task distributions. A-GEM also stores examples, but it uses them to form a gradient constraint. More memory does not automatically mean a better constraint, especially when the stored examples come from visually different tasks.

Third, buffer size is a practical issue. In real systems, memory is limited. A method should not only be judged by its best possible result with a large memory budget, but also by how well it performs when memory is smaller.

The buffer-size results are one of the clearest parts of the project. ER behaves predictably: larger buffers reduce forgetting and improve final accuracy in the datasets where forgetting is present. A-GEM does not show the same pattern. Its performance is flatter, and on CIFAR-100 the largest memory size becomes unstable. This suggests that direct sample replay is a more reliable way to use episodic memory than using memory only as a gradient constraint.

6. Main Results

Rotated MNIST

Method	ACC	BWT	LA	Forgetting
Vanilla	0.596 ± 0.003	-0.331 ± 0.003	0.894 ± 0.000	0.331 ± 0.003
EWC	0.607 ± 0.002	-0.317 ± 0.002	0.892 ± 0.000	0.317 ± 0.002
MAS	0.639 ± 0.002	-0.258 ± 0.003	0.870 ± 0.001	0.258 ± 0.003
OGD	0.591 ± 0.010	-0.333 ± 0.010	0.891 ± 0.000	0.333 ± 0.010
ER	0.833 ± 0.002	-0.059 ± 0.002	0.886 ± 0.001	0.059 ± 0.002
A-GEM	0.712 ± 0.003	-0.202 ± 0.003	0.893 ± 0.001	0.202 ± 0.003

Rotated MNIST gives the clearest evidence for replay. ER reaches 83.3% final accuracy and has much lower forgetting than the other methods. The learning accuracy values are similar across methods, so the main difference is not that ER learns the current task much better. The difference is that ER retains previous tasks much more effectively.

MAS is the best of the regularisation methods, but it is still far behind ER. OGD is close to Vanilla, which suggests that the gradient constraint does not help much in this smooth rotation setting.

EMNIST

Method	ACC	BWT	LA	Forgetting
Vanilla	0.839 ± 0.001	$+0.034 \pm 0.001$	0.812 ± 0.001	-0.034 ± 0.001
EWC	0.831 ± 0.001	$+0.029 \pm 0.001$	0.808 ± 0.001	-0.029 ± 0.001
MAS	0.877 ± 0.001	$+0.020 \pm 0.002$	0.861 ± 0.001	-0.020 ± 0.002
OGD	0.564 ± 0.006	$+0.069 \pm 0.009$	0.509 ± 0.006	-0.069 ± 0.009
ER	0.839 ± 0.002	$+0.034 \pm 0.001$	0.812 ± 0.001	-0.034 ± 0.001
A-GEM	0.843 ± 0.001	$+0.036 \pm 0.001$	0.814 ± 0.001	-0.036 ± 0.001

EMNIST is the exception to the replay story. Most methods have positive BWT, which means later tasks improve earlier performance rather than damaging it. In this case, replay has little advantage because there is not much forgetting to prevent.

MAS performs best here, reaching 87.7% ACC. My interpretation is that MAS may help preserve a useful shared representation across visually related transformations. OGD performs poorly because it seems to restrict learning too much; even though it does not show forgetting in the BWT metric, its learning accuracy is much lower.

This result is useful because it prevents the report from making an overly simple claim. Replay is strongest when there is real forgetting, but not every visual-drift setting produces forgetting.

CIFAR-100

Method	ACC	BWT	LA	Forgetting
Vanilla	0.174 ± 0.007	-0.511 ± 0.021	0.634 ± 0.021	0.511 ± 0.021
EWC	0.171 ± 0.008	-0.451 ± 0.074	0.577 ± 0.070	0.451 ± 0.074
MAS	0.403 ± 0.017	-0.091 ± 0.014	0.484 ± 0.012	0.091 ± 0.014
OGD	0.175 ± 0.005	-0.510 ± 0.054	0.635 ± 0.045	0.510 ± 0.054
ER	0.465 ± 0.010	-0.215 ± 0.008	0.658 ± 0.004	0.215 ± 0.008
A-GEM	0.414 ± 0.011	-0.279 ± 0.034	0.665 ± 0.031	0.279 ± 0.034

CIFAR-100 is the most difficult dataset because the tasks introduce different object categories. ER again gives the best final accuracy, reaching 46.5%. A-GEM and MAS are both competitive, but still lower than ER.

One interesting point is that MAS has lower forgetting than ER, but ER still has higher final accuracy. This suggests that MAS is stable, but less plastic: it protects previous knowledge, but does not learn the sequence as effectively. ER has a better balance between learning and retention.

EWC and OGD are close to Vanilla on CIFAR-100. This suggests that parameter protection and gradient projection are not enough in this stronger semantic-drift setting.

7. Buffer-Size Results

For ER, the trend is very clear:

Buffer Size	CIFAR-100 ACC	Rotated MNIST ACC	EMNIST ACC
200	0.337 ± 0.012	0.738 ± 0.006	0.839 ± 0.002
500	0.414 ± 0.009	0.799 ± 0.004	0.839 ± 0.002
1000	0.465 ± 0.010	0.833 ± 0.002	0.839 ± 0.002
2000	0.538 ± 0.012	0.859 ± 0.003	0.839 ± 0.002

On Rotated MNIST and CIFAR-100, increasing the buffer size improves ER consistently. This supports the idea that ER benefits from broader coverage of old data. On EMNIST, the buffer size makes almost no difference, which fits the earlier interpretation that EMNIST does not suffer much forgetting.

The A-GEM results are different. Increasing memory does not give the same steady improvement, and the largest memory size is unstable on CIFAR-100. This is important because ER and A-GEM both use memory, but they do not use it in the same way. The result suggests that the direct use of stored examples in ER is more robust than using stored examples only to define a gradient constraint.

8. Cost and Practical Considerations

I also tracked cost because accuracy alone is not enough to judge a continual learning method. In practice, a method has to be useful under limited time and memory.

There are two main cost dimensions in the current experiments:

- **Training time**, which measures how much computational overhead the method adds.
- **Extra storage**, which measures memory used by method-specific components such as replay buffers, Fisher matrices, parameter-importance estimates, or gradient memories.

This matters because different continual learning methods pay for stability in different ways. Replay methods store data. EWC stores importance information. OGD stores gradient-related information. A-GEM stores memory and also adds extra gradient computations. A method can look good in accuracy but become less attractive if it is very slow or memory-heavy.

Dataset	Method	Training Time	Extra Storage
CIFAR-100	Vanilla	243 ± 2 s	0.00 MB
CIFAR-100	EWC	694 ± 3 s	251.48 MB
CIFAR-100	MAS	327 ± 2 s	25.15 MB
CIFAR-100	OGD	283 ± 2 s	125.74 MB
CIFAR-100	ER	461 ± 0 s	11.73 MB
CIFAR-100	A-GEM	1139 ± 1 s	117.26 MB

Dataset	Method	Training Time	Extra Storage
Rotated MNIST	Vanilla	9 ± 1 s	0.00 MB
Rotated MNIST	EWC	42 ± 4 s	15.53 MB
Rotated MNIST	MAS	23 ± 2 s	1.55 MB
Rotated MNIST	OGD	18 ± 1 s	7.76 MB
Rotated MNIST	ER	23 ± 2 s	3.00 MB
Rotated MNIST	A-GEM	20 ± 0 s	29.98 MB
EMNIST	Vanilla	70 ± 6 s	0.00 MB
EMNIST	EWC	249 ± 21 s	8.02 MB
EMNIST	MAS	839 ± 75 s	1.60 MB
EMNIST	OGD	93 ± 9 s	1.60 MB
EMNIST	ER	69 ± 5 s	3.00 MB
EMNIST	A-GEM	139 ± 13 s	14.99 MB

The CIFAR-100 cost results are especially important. ER has the best final accuracy and only needs 11.73 MB of extra storage. EWC uses much more storage, 251.48 MB, because it stores parameter-importance information. OGD also uses substantial storage, 125.74 MB. A-GEM is the slowest method on CIFAR-100, taking 1139 seconds, because of the extra computations needed for gradient projection.

This makes ER attractive not only because it performs well, but because its cost is easy to understand and relatively modest. It does require storing examples, but in these experiments that storage is lower than several non-replay methods. It also improves predictably when more memory is available.

The cost analysis therefore supports a more practical conclusion: **ER is not just the most accurate method in the main drift settings; it is also one of the most efficient choices when considering storage, stability, and training overhead together.**

9. Current Interpretation

My current interpretation is:

Direct replay is highly effective when the model faces real visual drift and forgetting.

ER is stronger than A-GEM because it uses memory directly in the loss, while A-GEM uses memory indirectly through a gradient constraint.

MAS is the strongest regularisation method and is especially useful when tasks share structure, as in EMNIST.

EWC is less effective in these experiments, especially under semantic drift.

OGD seems too restrictive in several settings and can reduce plasticity.

EMNIST should be discussed carefully because it shows positive transfer rather than standard catastrophic forgetting.

Overall, the results support ER as the most reliable method for the main visual-drift settings, while also showing that the usefulness of a continual learning method depends on the type of drift.

10. Figures Included

The following figures are included after the written report:

- Overall cross-method summary
- Accuracy trends across all datasets
- Buffer/memory sensitivity by dataset
- Accuracy-cost trade-off
- Rotated MNIST multi-seed comparison
- CIFAR-100 multi-seed comparison

Key Experimental Figures

Figure 1. Overall cross-method thesis summary

Continual Learning under Visual Drift
Replay vs. Regularization vs. Optimization Methods (mean $\pm$ std, 3 seeds)

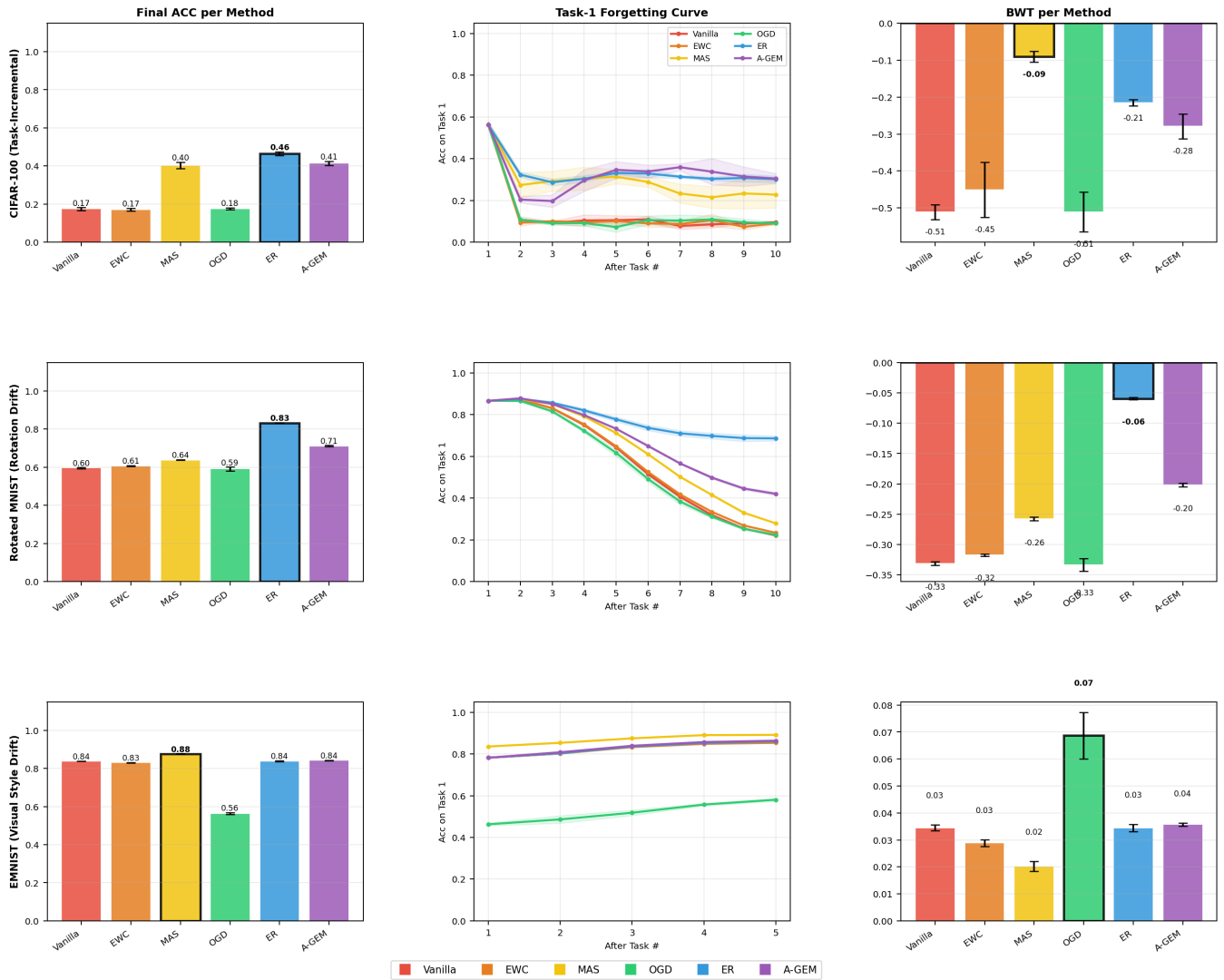


Figure 2. All-dataset accuracy trends

Continual Learning — All Datasets & Methods (mean $\pm$ std, 3 seeds)

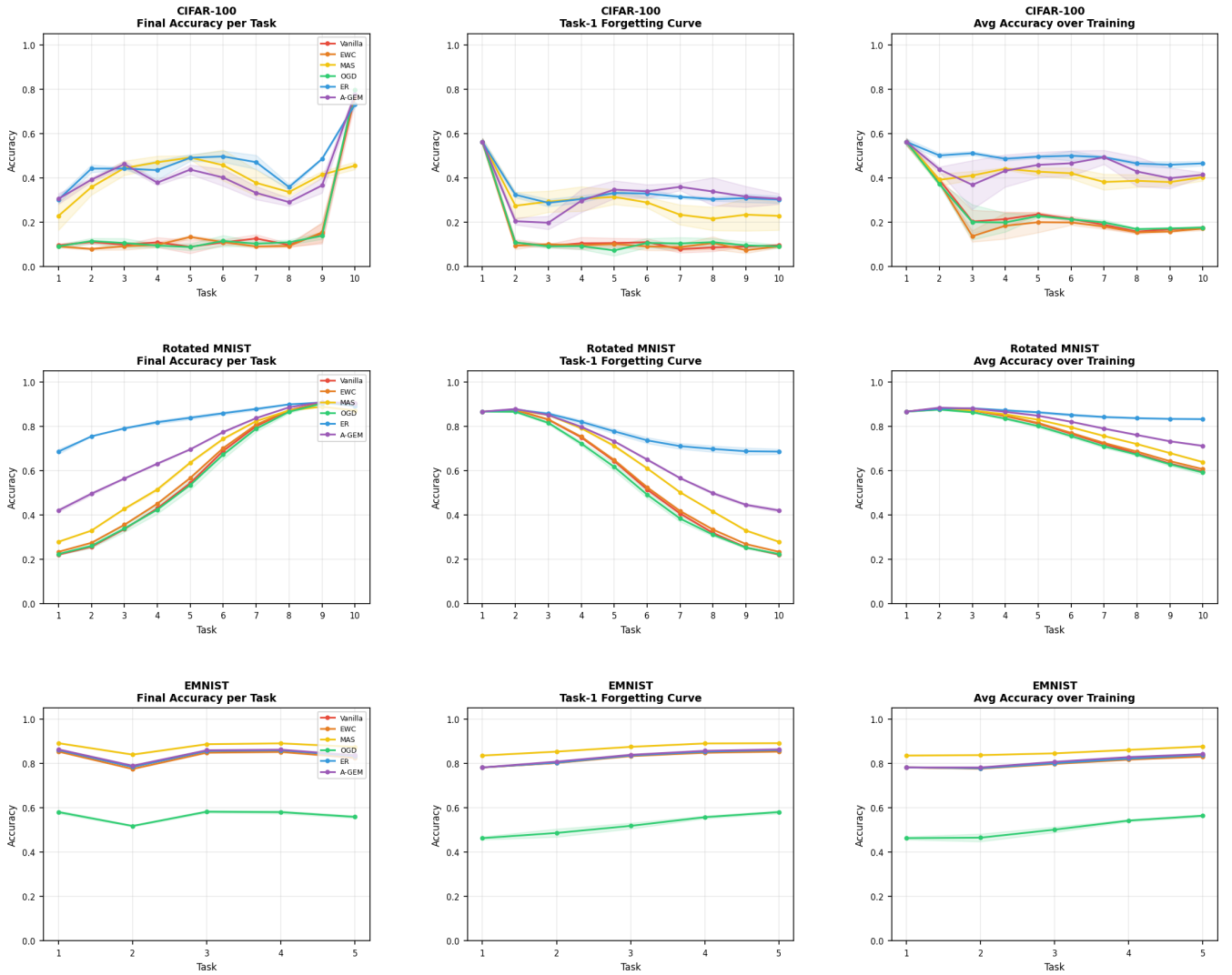


Figure 3. Buffer/memory sensitivity by dataset

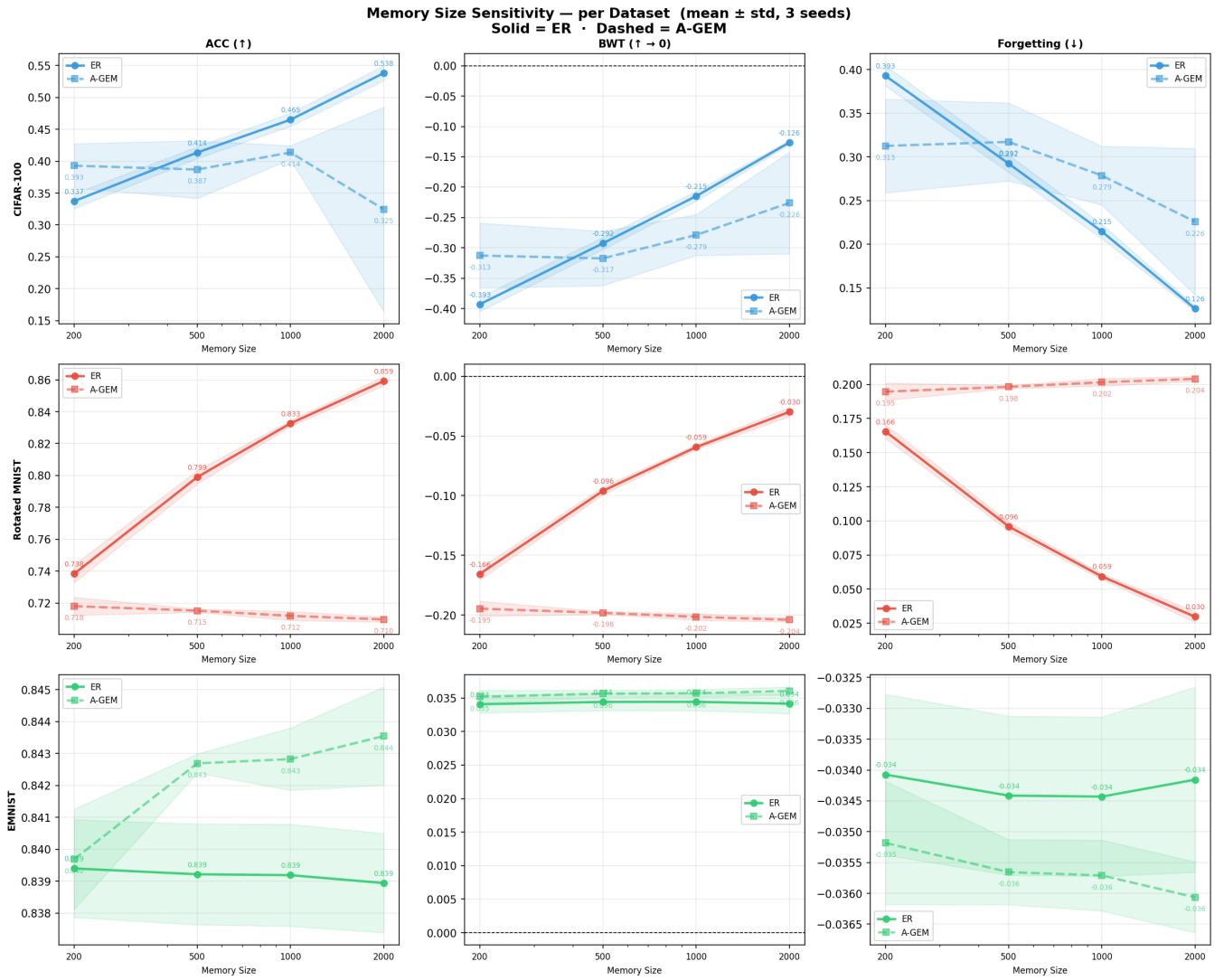


Figure 4. Accuracy-cost trade-off

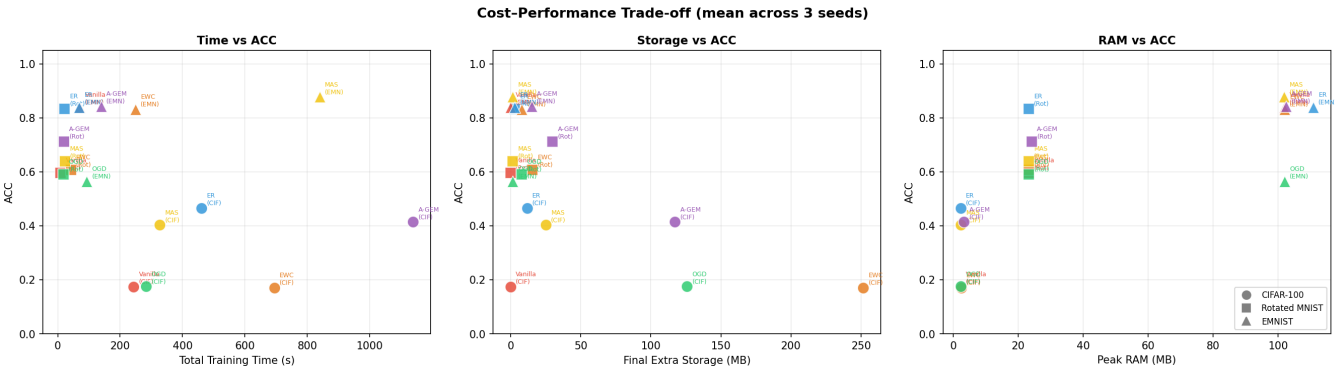


Figure 5. Rotated MNIST multi-seed comparison

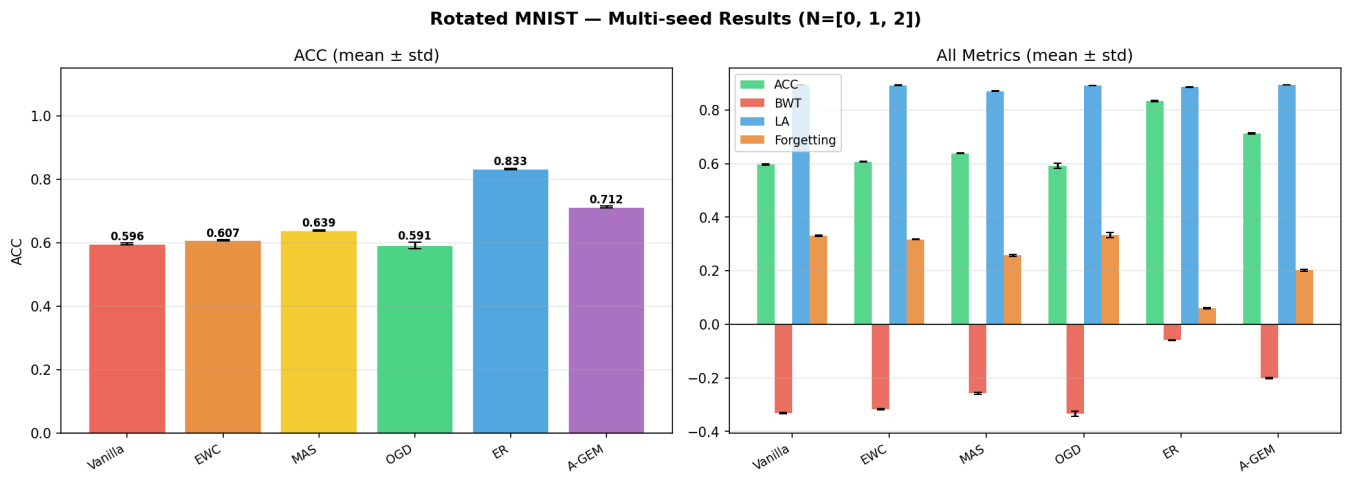


Figure 6. CIFAR-100 multi-seed comparison

